


```

function makeup_amount(x, coins) {
  if (x === 0) {
    return list(null);
  } else if (x < 0 || is_null(coins)) {
    return null;
  } else {
    // Combinations that do not use the head coin
    const combi_A = makeup_amount(x, tail(coins));

    // Combinations that do not use the head coin
    // for the remaining amount.
    const combi_B = makeup_amount(x - head(coins), tail(coins));

    // Combinations that use the head coin.
    const combi_C = map(x => pair(head(coins), x), combi_B);

    return append(combi_A, combi_C);
  }
}

function subsets(xs) {
  // if (is_null(xs)) {
  //   return list(null);
  // }
  // else {
  //   const no_current = subsets(tail(xs));
  //   const has_current = map(x => pair(head(xs), x), no_current);
  //   return append(no_current, has_current);
  // }

  return accumulate((x, y) => append(map(z => pair(x, z), y),
    y),
    list(null, xs);
}

```

```

12 function flatten_tree(tre) {
13   return is_null(tre)
14     ? null
15     : (!is_list(head(tre))
16       ? pair(head(tre), flatten_tree(tail(tre)))
17       : append(flatten_tree(head(tre)),
18               flatten_tree(tail(tre))));
19 }

function accumulate_tree(f, op, initial, tree) {
  return accumulate((x, y) => op((is_list(x)
    ? f(x)),
    y),
    initial,
    tree);
}

```

TREES →

```

function map_tree(f, tree) {
  return map(x => (is_list(x)
    ? map_tree(f, x)
    : f(x)),
    tree);
}

function filter_tree(pred, tree) {
  return filter(x => !is_null(x),
    accumulate((x, y) => is_list(x)
    ? pair(filter_tree(pred, x), y)
    : pred(x)
    ? pair(x, y)
    : y,
    null,
    tree));
}

```

```

function insert(bst, item) {
  return is_empty_tree(bst) // If space found, create item here
    ? make_tree(item, make_empty_tree(), make_empty_tree())
    : item < entry(bst) // Else search till empty space found
    ? make_tree(entry(bst), insert(left_branch(bst), item), right_branch(bst))
    : make_tree(entry(bst), left_branch(bst), insert(right_branch(bst), item));
}

```

```

1 const increment_repeater =
2   repeater => f => x => repeater(f)(f(x));
3
4 const twice = f => x => f(f(x));
5 const thrice = increment_repeater(twice);
6 const fourtimes = increment_repeater(thrice);
7 const warn = thrice(display);
8 warn("ALERT"); // should display "ALERT"
9 // three times in orange
10 const bigwarn = fourtimes(display);
11 bigwarn("A L E R T"); // should display "A L E R T"
12 // four times in orange
13 // (the REPL will display
14 // "A L E R T" a fifth time
15 // [in white] as the value
16 // returned by bigwarn)

```

```

1 const zero_repeater = f => x => x;
2 const one_repeater = f => x => f(zero_repeater, () => zero_repeater(f)(x));
3 const two_repeater = f => x => f(one_repeater, () => one_repeater(f)(x));
4 const three_repeater = f => x => f(two_repeater, () => two_repeater(f)(x));
5
6 const to_int = repeater => repeater((iter_count, x) => x() + 1)(0);
7
8 // Checks if the repeater is a zero_repeater. Runs in O(1) time.
9 const is_zero = repeater => repeater((iter_count, x) => false)(true);
10 const decrement_repeater = repeater => is_zero(repeater)
11   ? zero_repeater
12   : f => x => repeater((iter_count, x)
13     => iter_count)(x)(f)(x);
14
15 to_int(decrement_repeater(three_repeater)); // should return 2

```

```

function permutations(s) {
  if (is_null(s)) {
    return list(null);
  }
  else {
    return accumulate(append, null, // Flattens list
      map(x => {
        map(p => pair(x, p), //
          permutations(remove(x, s)));
      }, s));
  }
}

```

```

5 function insert(x, xs) {
6   return is_null(xs)
7     ? list(x)
8     : (x <= head(xs)
9       ? insert(x, tail(xs))
10      : pair(head(xs), insert(x, tail(xs)));
11 }
12
13 function insertion_sort(xs) {
14   return is_null(xs)
15     ? null
16     : insert(head(xs), insertion_sort(tail(xs)));
17 }

```

```

1 function selection_sort(xs) {
2   if (is_null(xs)) {
3     return null;
4   }
5   // We pick the smallest element, where should it go?
6   // What should we recurse on?
7   // YOUR SOLUTION HERE
8   return (x => pair(x, selection_sort(remove(x, xs)))) (smallest(xs));
9 }
10
11
12 function min(a, b) {
13   return a < b ? a : b;
14 }
15
16 // given a non-empty list xs, returns the smallest item in xs
17 function smallest(xs) {
18   return is_null(tail(xs))
19     ? head(xs)
20     : (x => head(xs) < x ? head(xs) : x) (smallest(tail(xs)));
21 }

```

Quick sort

```

3 function partition(xs, p) {
4   return pair(filter(x => x <= p, xs), filter(x => x > p, xs));
5 }
6
7 function quicksort(xs) {
8   if (is_null(xs) || is_null(tail(xs))) {
9     return xs;
10   }
11   const parts = partition(tail(xs), head(xs));
12   return append(quicksort(head(parts)), pair(head(xs), quicksort(tail(parts))));
13 }
14

```

Merge Sort.

```

1 /* Merges two pre-sorted lists x, y together. Returns the merged sorted list.
2   Runs in O(n) time. */
3 function merge(x, y) {
4   if (is_null(x)) {
5     return y;
6   }
7   else if (is_null(y)) {
8     return x;
9   }
10  else {
11    return head(x) <= head(y)
12      ? pair(head(x), merge(tail(x), y))
13      : pair(head(y), merge(x, tail(y)));
14  }
15 }
16
17 /* Splits xs into two sub-lists. Returns a pair of two lists of n and
18   (length(xs) - n) elements respectively for 0 <= n <= length(xs).
19   Runs in O(n) time. */
20 function split(xs, n) {
21   return n === 0
22     ? pair(null, xs)
23     : (1 => pair(pair(head(xs), head(1)), tail(1)))(split(tail(xs), n - 1));
24 }
25
26 /* Sorts the list xs and returns a pair of the sorted list and the total
27   number of swaps done. Runs in O(n log(n)) time. */
28 function sort(xs) {
29   // Base cases xs is empty, or with one element. Return xs and 0 total swaps.
30   if (is_null(xs) || is_null(tail(xs))) {
31     return xs;
32   }
33   else {
34     // Split xs into two sub-lists
35     const splitted = split(xs, math_floor(length(xs) / 2));
36
37     // Sort the two sub-lists
38     const x = sort(head(splitted));
39     const y = sort(tail(splitted));
40
41     // Merge the two sorted sub-lists together
42     const merger = merge(x, y);
43
44     return merger;
45   }
46 }

```

BST

```

1 const zero_repeater = f => x => x;
2 const one_repeater = f => x => f(zero_repeater, () => zero_repeater(f)(x));
3 const two_repeater = f => x => f(one_repeater, () => one_repeater(f)(x));
4 const three_repeater = f => x => f(two_repeater, () => two_repeater(f)(x));
5
6 const to_int = repeater => repeater((iter_count, x) => x() + 1)(0);
7
8 const increment_repeater = repeater => f => x => f(repeater, () => repeater(f)(x));
9
10 const add_repeater = (repeater1, repeater2) =>
11   repeater1((iter_count, x) => increment_repeater(x())
12     (repeater2));
13
14 to_int(add_repeater(two_repeater,
15   three_repeater)); // should return 5

```

Never
let
me
see
this.